

PaperMind

Privacy-First, Local-First Document Intelligence — Open Source NotebookLM Alternative

[MIT Licensed](#) · [Flutter Desktop \(Mac / Windows / Linux\)](#) · [FastAPI](#) · [pgvector](#) · [Ollama](#)

1. Executive Summary & Goals

1.1 The Problem

NotebookLM is the closest thing to a personal research assistant available today — but it requires a Google account, sends every document to Google’s servers, locks users to Gemini, and offers no offline mode. Researchers, students, and professionals working with sensitive or proprietary material cannot use it.

1.2 The Solution

PaperMind is a fully open-source, local-first document intelligence workspace that runs entirely on the user’s machine. Documents never leave the device unless the user explicitly provides a cloud API key. It supports:

- Any LLM provider via a unified pluggable interface — Ollama (local), OpenAI, Anthropic Claude, Google Gemini
- Multi-format document ingestion with high-precision cited answers
- Advanced retrieval: HyDE query expansion + cross-encoder re-ranking
- Multi-query decomposition for complex cross-document questions
- Deterministic structured artifact generation (flashcards, timelines, summaries)
- Full offline operation with zero cloud dependency when using Ollama

1.3 Why This Gets Used

The answer to “who uses this?” is direct: anyone who has tried NotebookLM and been blocked by privacy concerns, model lock-in, or Google account requirements. The open-source, self-hosted positioning means:

- Researchers and students can process confidential papers and notes locally
- Developers can self-host for their team without SaaS costs
- Contributors can extend it — new LLM providers, new source parsers, new artifact types
- GitHub stars compound as the community grows: every new provider plugin is a PR

1.4 Target Architecture

- **Flutter Desktop (Mac / Windows / Linux), BLoC + Clean Architecture:** Frontend
 - **FastAPI (Python), async ingestion and retrieval pipelines:** Backend
 - **PostgreSQL + pgvector (unified relational + vector store):** Storage
 - **Pluggable provider interface — Ollama, OpenAI, Claude, Gemini:** LLM Layer
 - **Single docker-compose.yml — one command to run everything locally:** Deployment
-

2. Core Epics & Functional Requirements

Epic 1 — Multi-Source Ingestion Pipeline

FR-1.1 Multi-Format Support

The backend **MUST** ingest .pdf (multi-page), .md, .txt, and .mp3/.m4a audio files. Audio sources are transcribed via OpenAI Whisper (local model) before entering the standard chunking pipeline, enabling lecture recordings and meeting notes as first-class sources.

FR-1.2 Asynchronous Chunking Engine

Documents **MUST** be split asynchronously using a recursive character text splitter with the following parameters:

- Chunk size: 500 tokens using tiktoken cl100k_base tokenizer (aligned to embedding model input)
- Overlap window: 50 tokens to preserve paragraph context boundaries
- Minimum chunk size: 100 tokens — sub-threshold trailing chunks are merged into the preceding chunk

FR-1.3 Immutable Lineage Metadata

Every text chunk **MUST** be tagged with an immutable metadata payload:

```
{ source_id: UUID, chunk_index: int (0-based),
  page_number: int | null, char_start: int,
  char_end: int, audio_timestamp_seconds: float | null }
```

FR-1.4 Atomic Dual-Write Transaction

The ingestion worker **MUST** save relational metadata to the sources table and vector embeddings to the document_chunks table within a single database transaction. Partial writes **MUST** roll back entirely.

Epic 2 — Pluggable LLM Provider Interface

This is the core differentiator of PaperMind over NotebookLM. All LLM interactions **MUST** route through a unified provider abstraction layer.

FR-2.1 Provider Abstraction

A single abstract LLMProvider class MUST define the interface. Concrete implementations: OllamaProvider, OpenAIProvider, ClaudeProvider, GeminiProvider. Adding a new provider requires implementing one class with no changes to business logic.

FR-2.2 Per-Notebook Provider Configuration

Each notebook stores its own LLM provider configuration in the database:

```
{ provider: 'ollama' | 'openai' | 'claude' | 'gemini',  
  model_name: string,          // e.g. 'llama3', 'gpt-4o', 'claude-sonnet-4-6'  
  api_key: string | null,      // null for Ollama (local)  
  base_url: string | null     // Ollama endpoint, default http://localhost:11434 }
```

FR-2.3 API Key Security

API keys MUST be stored encrypted at rest using Fernet symmetric encryption. Keys are decrypted in-memory only during inference and never logged or transmitted beyond the configured provider endpoint.

FR-2.4 Streaming Response via SSE

All LLM inference calls MUST return tokens via Server-Sent Events (SSE) from FastAPI. The Flutter client MUST render tokens progressively as they arrive. Complete-response buffering is not acceptable for any chat endpoint.

Epic 3 — Multi-Tenancy & Dynamic Context Toggling

FR-3.1 Multi-Document Notebook Associations

A single User Notebook MUST support up to 50 distinct source files concurrently. Source file size limit: 50MB per file.

FR-3.2 Dynamic Context Selection

Users MUST be able to toggle individual sources on/off per query. The active_source_ids array is sent with every query payload and enforced at the vector retrieval layer.

FR-3.3 Scoped Vector Search

The query handler MUST apply a hard SQL IN-clause filter on source_id before performing cosine similarity calculations. Global vector scans across non-selected sources are strictly prohibited. This ensures queries against a 50-source notebook with 2 sources active perform identically to a 2-source notebook.

Epic 4 — Advanced Retrieval Pipeline (Production RAG)

This is the technical depth layer that separates PaperMind from toy RAG implementations.

FR-4.1 HyDE — Hypothetical Document Embedding

Before embedding the user's raw query, the system MUST:

1. Send the query to the configured LLM with a HyDE prompt: "Write a short passage that would directly answer: {query}"
2. Embed the hypothetical answer (not the raw query) using the configured embedding model
3. Use this hypothetical embedding as the retrieval vector against pgvector

Rationale: short or vague user queries embed poorly. A hypothetical answer embedding occupies the same semantic space as actual relevant documents, dramatically improving retrieval recall for ambiguous queries.

FR-4.2 Multi-Query Decomposition

For queries exceeding 15 words or containing conjunctions (and, compare, versus, both), the system MUST:

4. Send the query to the LLM with a decomposition prompt requesting 2–3 atomic sub-queries in JSON format
5. Execute independent pgvector retrievals for each sub-query
6. Merge and deduplicate the retrieved chunks by chunk_id
7. Pass the unified chunk set to the re-ranker and then the LLM

Example: "Compare the authentication approach in doc A with the rate limiting strategy in doc B" decomposes to ["authentication approach", "rate limiting strategy"] — each retrieved independently, then merged.

FR-4.3 Hybrid Retrieval & Cross-Encoder Re-Ranking

The retrieval pipeline MUST execute the following sequence:

8. Dense vector retrieval: top 15 candidate chunks from pgvector using HNSW index
9. Cross-encoder re-ranking: pass (query, chunk) pairs through cross-encoder/ms-marco-MiniLM-L-6-v2 running in-process via sentence-transformers
10. Select top 4 re-ranked chunks for LLM context injection

The cross-encoder runs locally and adds approximately 80ms latency for 15 candidates on CPU — acceptable for desktop use.

FR-4.4 Adaptive Similarity Threshold & Web Fallback

Each notebook stores a calibrated similarity_threshold (default: 0.70, range: 0.50–0.90). If the maximum similarity score from pgvector falls below this threshold AND enable_web_fallback is active:

- Route query to Tavily or Serper web search API
- Return combined answer clearly labelled with `source_type_used: 'web'`
- Display banner: “Information retrieved from open web. Save as a markdown source in this notebook?”
- Threshold is user-adjustable per notebook to handle domain-specific corpora

FR-4.5 Conversation Memory

Each notebook session maintains a `conversation_turns` table storing prior Q&A pairs. The last 3 turns **MUST** be included in every LLM prompt as chat history. This enables follow-up questions (“expand on point 2”) without re-stating context.

Epic 5 — Deterministic Structured Artifact Generation

FR-5.1 Guaranteed JSON Schema via Tool Calling

All artifact generation **MUST** use LLM structured tool-calling / function-calling APIs (or Ollama’s JSON mode) to guarantee schema-valid output. Natural language parsing of artifact content is explicitly prohibited.

FR-5.2 Flashcard Schema

```
{ "cards": [  
  { "front": string,  
    "back": string,  
    "source_chunk_ids": List[UUID],  
    "difficulty": "easy" | "medium" | "hard" }  
]
```

FR-5.3 Timeline Schema

```
{ "timeline": [  
  { "date_or_period": string,  
    "event_description": string,  
    "source_chunk_ids": List[UUID] }  
]
```

FR-5.4 Executive Summary Schema

```
{ "summary": { "tldr": string,  
               "key_findings": List[string],  
               "open_questions": List[string],  
               "source_coverage": List[UUID] }  
}
```

FR-5.5 Artifact Caching

Generated artifacts **MUST** be persisted in an artifacts table keyed on (notebook_id, artifact_type, source_hash). The source_hash is a SHA-256 of the active source IDs + document content versions. Cache is invalidated automatically when sources change. This prevents duplicate token billing for unchanged notebooks.

3. Embedding Model Configuration

The embedding model is configurable per deployment. Default recommendations:

Model	Provider	Dimensions	Notes
text-embedding-3-small	OpenAI API	1536	Best quality, requires API key
all-MiniLM-L6-v2	Local (sentence-transformers)	384	Fully offline, good quality
nomic-embed-text	Ollama (local)	768	Best local option, no API key needed

The embedding model **MUST** be fixed per notebook at creation time. Changing the model on an existing notebook requires re-ingestion of all sources (enforced by the backend with a validation error).

4. API Data Contracts

POST /api/v1/chat/query

```
{
  "notebook_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "message": "What hardware specs are required for local execution?",
  "active_source_ids": ["7b11c9f0-...", "a23d56f1-..."],
  "enable_web_fallback": true,
  "stream": true
}
```

SSE stream format (when stream: true):

```
data: {"type": "token", "content": "According"}
data: {"type": "token", "content": " to"}
data: {"type": "citations", "citations": [{...}]}
data: {"type": "done", "source_type_used": "local_documents"}
```

POST /api/v1/sources/upload

```
multipart/form-data:
  notebook_id: UUID
  file: binary (pdf | md | txt | mp3 | m4a)

Response 202 Accepted:
{ "source_id": UUID, "status": "ingestion_queued",
  "estimated_chunks": int }
```

POST /api/v1/artifacts/generate

```
{
  "notebook_id": "UUID",
  "artifact_type": "flashcards" | "timeline" | "summary",
  "active_source_ids": ["UUID", ...]
}
```

GET /api/v1/notebooks/{id}/artifacts

```
Response 200 OK:
{ "artifacts": [
  { "artifact_id": UUID,
    "artifact_type": string,
    "created_at": ISO8601,
    "cache_hit": bool,
    "payload": { ...schema-specific JSON... } }
]
```

5. BLoC State Machine Matrix

All Flutter UI state transitions MUST conform to this matrix. No UI element may reflect a state not defined here.

BLoC Component	Triggering Event	Processing Action	Output UI State
NotebookSourceBloc	UploadDocumentEvent	POST /sources/upload with multipart stream	SourceUploadInProgress (progress bar) → SourceUploadSuccess (appended to sidebar)
NotebookSourceBloc	ToggleSourceEvent	Mutate active_source_ids list in memory only	SourceSelectionUpdated (instant checkbox toggle, no API call)

NotebookChatBloc	SubmitMessageEvent	POST /chat/query — open SSE stream	ChatResponseLoading (locks input, shows typing indicator) → ChatResponseStreaming (renders tokens as received) → ChatResponseSuccess (citations rendered)
NotebookChatBloc	WebFallbackEvent	Triggered on source_type_used: 'web' in SSE done event	WebFallbackBannerVisible (Save to notebook? banner above response)
ArtifactEngineBloc	GenerateFlashcardsEvent	POST /artifacts/generate — artifact_type: flashcards	ArtifactGenerationLoading (shimmer placeholders) → ArtifactFlashcardDeckReady (swipeable deck)
ArtifactEngineBloc	GenerateTimelineEvent	POST /artifacts/generate — artifact_type: timeline	ArtifactGenerationLoading → ArtifactTimelineReady (scrollable chronological list)
LLMProviderBloc	ConfigureProviderEvent	Validate API key / Ollama connectivity via health check	ProviderValidating → ProviderReady (green indicator) ProviderError (actionable message)

6. Database Schema

6.1 Core Tables

```

notebooks (id UUID PK, name TEXT, created_at TIMESTAMP,
           llm_provider TEXT, model_name TEXT,
           api_key_encrypted TEXT, base_url TEXT,
           embedding_model TEXT, similarity_threshold FLOAT DEFAULT 0.70)

sources (id UUID PK, notebook_id UUID FK, name TEXT,
         file_type TEXT, size_bytes INT,
         ingestion_status TEXT, created_at TIMESTAMP)

document_chunks (id UUID PK, source_id UUID FK,
                 chunk_index INT, page_number INT,
                 char_start INT, char_end INT,
                 audio_timestamp_seconds FLOAT,
                 content TEXT,
                 embedding VECTOR(1536))

conversation_turns (id UUID PK, notebook_id UUID FK,
                   role TEXT, content TEXT,
                   created_at TIMESTAMP)

artifacts (id UUID PK, notebook_id UUID FK,
           artifact_type TEXT, source_hash TEXT,

```



```
payload JSONB, created_at TIMESTAMP)
```

6.2 pgvector Index

```
-- HNSW index capped at 512MB working memory
SET hnsw.ef_construction = 64;
CREATE INDEX ON document_chunks
  USING hnsw (embedding_vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

7. Non-Functional Requirements

ID	Constraint	Specification
NFR-7.1	HNSW Memory Cap	pgvector HNSW index capped at 512MB RAM in Docker. Flat scan fallback if index not built.
NFR-7.2	SSE Streaming Latency	First token MUST appear within 2 seconds of query submission on localhost Ollama.
NFR-7.3	Error Decoupling	Flutter layer NEVER receives raw stack traces. All 4xx/5xx map to typed Failure model objects with client-readable messages.
NFR-7.4	Clean Architecture Isolation	DB column renames require changes only in the data-layer mapping models. Presentation layer has zero awareness of schema terminology.
NFR-7.5	Offline Operation	With Ollama + nomic-embed-text configured, the entire application MUST function with no internet connection.
NFR-7.6	One-Command Setup	docker-compose up MUST start PostgreSQL+pgvector, FastAPI backend, and run DB migrations. User installs Ollama separately via documented one-liner.

8. Bottom-Up Build Checklist

Execute strictly in order. Do not begin a step until the previous step's output is verified working.

Phase 1 — Infrastructure

1. Write docker-compose.yml: PostgreSQL 16 + pgvector extension, FastAPI service, volume mounts, environment variable injection for API keys
2. Write init.sql: create all tables, HNSW index, test vector insertion and cosine similarity query. Verify pgvector extension loads.
3. Install Ollama locally, pull llama3 and nomic-embed-text. Verify /api/generate and /api/embeddings endpoints respond.

Phase 2 — Backend Core

4. Implement LLMProvider abstract class + OllamaProvider. Write unit test: embed a sentence, verify vector shape.
5. Implement ingestion pipeline: file upload → chunking (tiktoken) → embed → dual-write transaction. Test with a 10-page PDF.
6. Implement HyDE query expansion. Unit test: compare retrieval results with raw query vs HyDE-expanded query on same document.
7. Implement multi-query decomposition. Unit test: verify a compound question decomposes to 2+ sub-queries.
8. Implement cross-encoder re-ranker with ms-marco-MiniLM-L-6-v2. Benchmark: measure latency for 15-candidate re-ranking on CPU.
9. Wire full /chat/query endpoint with SSE streaming. Test with curl, verify token-by-token output.
10. Add OpenAIProvider and ClaudeProvider. Verify same query produces valid output across all three providers with API keys.

Phase 3 — Flutter Frontend

11. Define domain entities and repository interfaces (Clean Architecture data layer). No widgets yet.
12. Implement all BLoC classes and state transitions from the matrix in Section 5. Unit test each state transition.
13. Build source sidebar: upload button, progress bar, source list with toggle checkboxes.
14. Build chat panel: SSE token streaming renderer, citation chip widgets, web fallback banner.
15. Build artifact panel: flashcard swipe deck, timeline list, provider config modal.

Phase 4 — Deployment & Open Source Launch

16. Write CONTRIBUTING.md: how to add a new LLM provider (one class, one PR).
17. Write README.md: one-command setup, screenshots, feature comparison table vs NotebookLM.
18. Run retrieval eval: index 3 test documents, run 20 benchmark questions, measure precision@4 (target: >0.75).
19. Package Flutter desktop binaries for Mac, Windows, Linux via GitHub Actions. Attach to GitHub Release.
20. Post to Hacker News Show HN, r/LocalLLaMA, r/selfhosted. The privacy + local-first + open-source angle resonates strongly in these communities.

9. Interview Talking Points

For every role this project covers, here is the answer to “what does this teach you?”

Interviewer Role	Your Answer
SDE (Backend)	Async ingestion pipeline, transactional dual-writes, SSE streaming, pluggable provider pattern, Docker orchestration
AI/ML Engineer	HyDE query expansion, cross-encoder re-ranking, multi-query decomposition, embedding model selection, RAG evaluation (precision@k)
Systems / Infra	pgvector HNSW index tuning, 512MB memory budget enforcement, one-command Docker setup, GitHub Actions CI/CD
Product / Full-Stack	Real users (open source community), privacy-first positioning, NotebookLM feature gap analysis, provider configurability as a product decision

To the question “do people actually use this?”: open source projects with a clear alternative narrative (privacy-first vs NotebookLM) attract real users fast — especially on r/LocalLLaMA and Hacker News where the local-first AI community is large and active. GitHub stars, issues, and PRs are measurable proof of adoption you can show in any interview.